

Image Enhancement with Deep Learning: A Comparative Study of CNN-Based Image Restoration Methods

Giuseppe Bellamacina, Daniele Barbagallo, Salvo Iurato, Mattia Campanella
Department of Mathematics and Computer Science
University of Catania, Italy
Deep Learning Course — A.Y. 2025/2026
Professors: G.M. Farinella, G. Ragusa

Abstract—We present a comprehensive comparative study of convolutional neural network architectures for image restoration under severe Gaussian noise ($\sigma = 100$). Seven distinct architectures are implemented within a unified, modular framework: UNet, UNet Residual, Attention UNet, DnCNN, Denoising Autoencoder, RIDNet, and Pix2Pix (conditional GAN). All models are trained and evaluated on the DIV2K dataset using consistent degradation, training infrastructure, and evaluation protocols. Our experiments demonstrate that RIDNet with 8 Enhancement Attention Blocks achieves the best overall performance (PSNR = 25.83 dB, SSIM = 0.725), followed closely by DnCNN (PSNR = 25.60 dB). We perform extensive ablation studies on architectural choices including upsampling strategies, attention mechanisms, loss function compositions, and training hyperparameters. Results show that residual learning, channel attention, and appropriate gradient clipping are critical factors for high-quality denoising under heavy noise conditions.

Index Terms—Image denoising, convolutional neural networks, residual learning, attention mechanisms, image restoration, deep learning

I. INTRODUCTION

Image denoising is a fundamental problem in low-level computer vision, aiming to recover a clean image from its noisy observation. With the advent of deep learning, CNN-based approaches have largely superseded traditional methods such as BM3D and NLM, achieving state-of-the-art results across a variety of noise models and intensity levels [3].

This work presents a systematic comparison of multiple deep learning architectures for image restoration. Rather than proposing a single novel method, we focus on implementing a *unified experimental framework* that enables fair, reproducible comparisons across fundamentally different architectural paradigms: encoder-decoder networks with skip connections (UNet family), purely sequential residual networks (DnCNN), attention-augmented architectures (RIDNet, Attention UNet), and generative adversarial networks (Pix2Pix).

All models are evaluated under identical conditions: the same dataset (DIV2K [1]), the same degradation model (additive Gaussian noise with $\sigma = 100$, a deliberately severe setting), and the same evaluation protocol (sliding-window inference with Hann blending on full-resolution images).

A. Contributions

The main contributions of this work are:

- A modular, extensible framework covering the complete pipeline from degradation generation through training to evaluation;
- A fair comparison of seven architectures under controlled conditions, with extensive ablation studies;
- Analysis of critical design choices: residual learning vs. direct prediction, bilinear vs. transposed convolution upsampling, batch normalization vs. group normalization, and multi-component loss functions.

B. Work Division

The project was developed collaboratively:

- **Giuseppe Bellamacina** — UNet, UNet Residual, Attention UNet, and the shared infrastructure (degradations, training pipeline, evaluation, utilities);
- **Salvo Iurato** — DnCNN and RIDNet;
- **Mattia Campanella** — Denoising Autoencoder;
- **Daniele Barbagallo** — Pix2Pix, GAN, NAFNet, and Swin2SR.

C. Paper Organization

Section II describes the degradation models. Section III details the shared training infrastructure. Section IV presents the model architectures. Section V discusses training procedures and hyperparameters. Section VI defines the evaluation methodology. Section VII reports experimental results with ablation studies and visual comparisons. Section VIII concludes the paper.

II. DEGRADATION MODELS

Degradations are the starting point of the restoration pipeline: they transform clean images into corrupted versions that models must learn to restore. Three types of parametric, reproducible degradations were implemented.

A. Gaussian Noise

Gaussian noise is the primary degradation used in our experiments. Given a pixel value x , the corrupted image is obtained as:

$$\tilde{x} = \text{clip}(x + \mathcal{N}(0, \sigma^2), 0, 255) \quad (1)$$

where σ controls noise intensity. We use $\sigma = 100$, a deliberately high value that severely corrupts image content and challenges the reconstruction capacity of all models. For reproducibility, each image receives a unique seed computed as $\text{seed} + \text{image_index}$.

B. Quantization and Dithering

Quantization reduces the color depth of an image, mapping the original 256 levels per channel to $L = 2^b$ levels where b is the number of bits per channel.

1) *Simple Quantization*: Direct depth reduction:

$$q(x) = \text{round}\left(\frac{x}{255} \cdot (L - 1)\right) \cdot \frac{255}{L - 1} \quad (2)$$

2) *Random Dithering*: Before quantization, uniform noise is added in the interval $[-s/2, s/2]$ where $s = 255/(L - 1)$ is the quantization step:

$$\tilde{x} = q\left(x + \mathcal{U}\left(-\frac{s}{2}, \frac{s}{2}\right) \cdot \text{noise_strength}\right) \quad (3)$$

This breaks the banding patterns typical of simple quantization.

3) *Ordered Dithering (Bayer)*: Uses Bayer matrices (2×2 , 4×4 , or 8×8) as spatially-varying thresholds. The Bayer matrix is normalized to $[-0.5, 0.5]$ and added to the image before quantization, producing regular, visually pleasing patterns.

4) *Floyd-Steinberg Dithering*: An error diffusion algorithm [11] that processes the image pixel by pixel. The quantization error $e = x - q(x)$ is distributed to adjacent unprocessed pixels according to the kernel:

$$\begin{bmatrix} \cdot & \cdot & \cdot \\ \cdot & * & \frac{7}{16} \\ \frac{3}{16} & \frac{5}{16} & \frac{1}{16} \end{bmatrix} \quad (4)$$

where $*$ denotes the current pixel. This produces perceptually superior results at the cost of sequential processing.

C. Salt-and-Pepper Noise

Salt-and-pepper noise replaces a fraction p of pixels with extreme values: the parameter `salt_vs_pepper` controls the ratio between white pixels (255, *salt*) and black pixels (0, *pepper*).

D. Automated Dataset Generation

The module `generate_degraded_dataset.py` provides batch processing of entire image directories with automatic path generation following a consistent naming convention (e.g., `data/degraded/gaussian/sigma_100/DIV2K_train_HR/`). Generation is skipped if the degraded dataset already exists, ensuring reproducibility without redundant computation.

III. TRAINING INFRASTRUCTURE

The training infrastructure is shared across all models and comprises the PyTorch dataset, the training pipeline, the experiment management system, loss functions, and evaluation utilities.

A. Dataset and Data Loading

The `ImageEnhancementDataset` class extends `torch.utils.data.Dataset` and manages loading of (degraded, clean) image pairs with random patch extraction.

1) *Patch Extraction*: Each image is divided into square patches of configurable size (typically 128×128 or 256×256). For each image, `patches_per_image` patches are extracted at random positions, so the effective dataset length is:

$$|\mathcal{D}| = N_{\text{images}} \times \text{patches_per_image} \quad (5)$$

Images are cached via `@lru_cache(maxsize=64)` to minimize disk I/O.

2) *Data Augmentation*: During training, geometric transformations are applied jointly to the degraded/clean pair via `Albumentations`:

- `HorizontalFlip` ($p = 0.5$)
- `VerticalFlip` ($p = 0.5$)
- `RandomRotate90` ($p = 0.5$)

Normalization maps pixels from $[0, 255]$ to $[-1, 1]$ via $x' = (x/255 - 0.5)/0.5$. During validation, only normalization is applied.

3) *DataLoader Configuration*: `DataLoaders` are configured with `pin_memory=True` for asynchronous GPU transfers, `persistent_workers=True` to avoid worker respawning, and `drop_last=True` during training for constant batch sizes. A worker initialization function disables OpenCV threading and sets per-worker NumPy seeds.

B. Loss Functions

Three loss functions of increasing complexity are available.

1) *Combined Loss* ($L1 + SSIM$):

$$\mathcal{L}_{\text{combined}} = \alpha \cdot \mathcal{L}_{L1} + \beta \cdot (1 - \text{SSIM}) \quad (6)$$

with default weights $\alpha = 0.84$, $\beta = 0.16$. Images are converted from $[-1, 1]$ to $[0, 1]$ for SSIM computation. For numerical stability under mixed precision (FP16), epsilon clamping and small Gaussian noise injection are applied to the SSIM computation.

2) *VGG Perceptual Loss*: The VGG Perceptual Loss [7] uses a pretrained VGG-16 as a feature extractor. Features are extracted at configurable layers (`relu2_2`, `relu3_3` by default) and the loss is computed as weighted L1 distance in feature space:

$$\mathcal{L}_{\text{perceptual}} = \sum_{l \in \text{layers}} w_l \cdot \|\phi_l(\hat{x}) - \phi_l(x)\|_1 \quad (7)$$

where ϕ_l denotes VGG-16 features at layer l . VGG parameters are frozen and inputs are normalized with ImageNet statistics.

3) *Combined Perceptual Loss*: Unifies all three components:

$$\mathcal{L} = \alpha \cdot \mathcal{L}_{L1} + \beta \cdot (1 - \text{SSIM}) + \gamma \cdot \mathcal{L}_{\text{perceptual}} \quad (8)$$

Typical weights are $\alpha = 0.6$, $\beta = 0.25$, $\gamma = 0.15$. When $\gamma = 0$, the VGG network is not loaded into memory.

C. Experiment Management

Each experiment is saved in a timestamped directory:

```
experiments/{model}/{degradation}/{timestamp}_{name}/
  checkpoints/    config.json  history.json
  samples/        logs/
```

The system supports automatic *forking*: if the configuration changes relative to the previous experiment, a new directory is created with its own metadata while copying the best checkpoint and training history from the parent. This enables systematic hyperparameter exploration with full traceability.

D. Checkpointing

Checkpoints save the complete training state: model weights, optimizer and scheduler states, current epoch, and metrics. At loading time, integrity validation checks all floating-point tensors for NaN/Inf values. In case of OOM errors during training, an emergency checkpoint is automatically saved before cleanup.

E. Telegram Notifications

A notification module sends training status updates (start, periodic progress, completion, early stopping, OOM errors) via Telegram Bot API, enabling remote monitoring of long-running experiments.

IV. MODEL ARCHITECTURES

This section describes the architectures implemented. All models operate on RGB images normalized to $[-1, 1]$ and produce output in the same range. Two paradigms are distinguished: **direct prediction** of the clean image and **residual learning**, where the model predicts the noise to be subtracted from the input.

A. UNet

Developed by: Giuseppe Bellamacina

The UNet [2] is an encoder-decoder architecture with skip connections, originally proposed for biomedical segmentation and adapted here to image restoration. The model learns a direct mapping from the degraded image to the clean one.

1) *Architecture*: The encoder consists of 4 downsampling stages, each comprising a DoubleConv block (two 3×3 convolutions with BatchNorm and ReLU) followed by 2×2 MaxPool:

$$\text{inc}(3 \rightarrow 64) \rightarrow d_1(64 \rightarrow 128) \rightarrow d_2(128 \rightarrow 256) \rightarrow d_3(256 \rightarrow 512) \rightarrow d_4(512 \rightarrow 1024) \quad (9)$$

The decoder reconstructs spatial resolution via upsampling. Two modes are supported:

- **Bilinear**: `nn.Upsample(scale_factor=2, mode='bilinear')` followed by DoubleConv.
- **Transposed Convolution**: `ConvTranspose2d(kernel=2, stride=2)`.

At each decoder level, encoder features are concatenated via skip connections. The final output is a 1×1 convolution mapping from 64 to 3 channels, without activation.

B. UNet Residual

Developed by: Giuseppe Bellamacina

The UNet Residual shares the same encoder-decoder architecture as the base UNet, with a fundamental difference in the learning paradigm: the model predicts the **noise** present in the image rather than the clean image. The denoised output is obtained by subtraction:

$$\hat{x}_{\text{clean}} = x_{\text{noisy}} - f_{\theta}(x_{\text{noisy}}) \quad (10)$$

where f_{θ} is the network predicting the noise component.

1) *Anti-Checkerboard*: In the transposed convolution variant, the kernel size was increased from 2×2 to 4×4 with padding=1, ensuring more uniform coverage of output pixels and eliminating the checkerboard artifacts typical of standard deconvolution.

C. Attention UNet

Developed by: Giuseppe Bellamacina

The Attention UNet extends the UNet with *attention gates* [6] in the skip connections, allowing the decoder to focus on the most relevant regions of encoder features.

1) *Attention Gate*: The attention gate computes a spatial attention map from the gating signal g (from the lower decoder level) and the skip signal x (from the encoder):

$$\alpha = \sigma(\psi(\text{ReLU}(W_g \cdot g + W_x \cdot x))) \quad (11)$$

where W_g and W_x are 1×1 convolutions with normalization, ψ is a 1×1 convolution reducing to a single channel, and σ is the sigmoid function. Skip features are modulated element-wise: $\hat{x} = \alpha \odot x$.

2) *Attention Levels*: Two configurations are supported:

- ‘‘bottleneck’’: attention gate applied only to the first decoder level.
- ‘‘all’’: attention gates applied at all decoder levels.

3) *Normalization*: Both BatchNorm2d and GroupNorm (with $\min(8, C)$ groups) are supported, selectable via the norm parameter.

4) *Adaptive Blending*: An optional blending mechanism moderates model intervention based on estimated noise intensity. If a `noise_sigma` value is provided at inference:

$$\alpha = \min\left(\frac{\sigma_{\text{noise}}}{\sigma_{\text{trained}}}, 1.0\right), \quad \text{output} = \alpha \cdot x_{\text{input}} + (1 - \alpha) \cdot x_{\text{denoised}} \quad (12)$$

D. DnCNN

Developed by: Salvo Iurato

DnCNN [3] (*Denoising Convolutional Neural Network*) is a purely sequential architecture exploiting residual learning. Unlike encoder-decoder architectures, it uses no downsampling or skip connections.

1) **Architecture:** The network consists of D convolutional layers (17 in the best configuration):

- 1) **First layer:** Conv2d($3 \rightarrow F$, 3×3) + ReLU (no BatchNorm).
- 2) **Layers 2 to $D-1$:** Conv2d($F \rightarrow F$, 3×3) + BatchNorm + ReLU.
- 3) **Last layer:** Conv2d($F \rightarrow 3$, 3×3) (no BatchNorm or activation).

The predicted noise is subtracted from the input: $\hat{x}_{\text{clean}} = x_{\text{noisy}} - f_{\theta}(x_{\text{noisy}})$. Output is clamped to $[-1, 1]$ at inference. Weights are initialized with Kaiming normal for convolutions and constants (1/0) for BatchNorm.

E. Denoising Autoencoder

Developed by: Mattia Campanella

The Denoising Autoencoder is an encoder-decoder architecture **without skip connections**, designed as a baseline for comparison with more advanced architectures.

1) **Architecture:** The **encoder** performs 4 downsampling stages via strided convolutions (3×3 , stride=2), each followed by BatchNorm and LeakyReLU(0.2):

$$3 \xrightarrow{s_2} 128 \xrightarrow{s_2} 64 \xrightarrow{s_2} 32 \xrightarrow{s_2} 16 \text{ (spatial)} \quad (13)$$

The **decoder** reconstructs resolution via Upsample(bilinear) + Conv2d(3×3) at each stage, with BatchNorm and LeakyReLU, followed by a final Tanh activation producing output in $[-1, 1]$.

The absence of skip connections forces the bottleneck to encode all information necessary for reconstruction, making the model more “lossy” but simpler, and serving as a lower bound for architectures with skip connections.

F. RIDNet

Developed by: Salvo Iurato

RIDNet [4] (*Real Image Denoising Network*) combines dilated convolutions and channel attention [9] to capture multi-scale information without pooling operations.

1) **Enhancement Attention Block (EAB):** The fundamental building block consists of:

- 1) Three convolutions with increasing dilation ($d = 1, 2, 3$), each followed by BatchNorm and ReLU, expanding the receptive field without reducing spatial resolution.
- 2) A ChannelAttention module (Squeeze-and-Excitation): Global Average Pooling $\rightarrow$ Conv 1×1 ($C \rightarrow C/r$) $\rightarrow$ ReLU $\rightarrow$ Conv 1×1 ($C/r \rightarrow C$) $\rightarrow$ Sigmoid, with reduction factor $r = 16$.
- 3) A local residual connection with scaling factor:

$$\text{EAB}(x) = x + s \cdot \text{CA}(\text{dilated_branch}(x)) \quad (14)$$

where s is the residual_scale (0.1 in our best experiments).

2) Full Architecture:

- 1) **Feature extraction:** Conv2d($3 \rightarrow F$, 3×3) + ReLU.
- 2) **Enhancement:** Sequence of N EAB blocks ($N = 8$ in best config).
- 3) **Reconstruction:** Conv2d($F \rightarrow 3$, 3×3) predicting noise.
- 4) **Global residual:** $\hat{x} = x - f_{\theta}(x)$.

Output is clamped to $[-1, 1]$ at inference.

G. Pix2Pix

Developed by: Daniele Barbagallo

Pix2Pix [5] is an image-to-image translation framework based on conditional GANs. The system comprises a **generator** and a **discriminator** trained adversarially.

1) **Generator:** The generator has a UNet-like architecture with 7 downsampling levels, designed for 256×256 inputs.

Encoder (6 layers + bottleneck): Each layer uses Conv2d(4×4 , stride=2) + BatchNorm + LeakyReLU(0.2). The first layer omits BatchNorm.

Decoder (6 layers + final): ConvTranspose2d(4×4 , stride=2) + BatchNorm + ReLU, with Dropout(0.5) in the first 3 layers. Skip connections via concatenation with corresponding encoder features. The final layer produces 3 channels with Tanh activation (or without, for residual learning).

2) **PatchGAN Discriminator:** The discriminator operates on local image patches, classifying 70×70 regions as real or generated. Input is the concatenation of the conditional image (degraded) and target/generated image (6 channels total). The architecture consists of 4 convolutional blocks (4×4 , progressively reduced stride) producing single-channel logit output (no sigmoid).

3) **Learning Mode:** In our experiments, the generator operates in **residual learning** mode: it predicts $\hat{n} = G(x_{\text{noisy}})$ and the clean image is obtained as:

$$\hat{x} = \text{clamp}(x_{\text{noisy}} - \hat{n}, -1, 1) \quad (15)$$

Predicted noise is additionally clamped to ± 0.6 for stability.

H. NAFNet

Developed by: Daniele Barbagallo

[TODO — Daniele]: NAFNet [12] (Nonlinear Activation Free Network) architecture description and implementation details to be added.

I. Swin2SR

Developed by: Daniele Barbagallo

[TODO — Daniele]: Swin2SR [13] (SwinV2 Transformer for image restoration) architecture description and implementation details to be added.

Model	Type	Residual	Attention	Output
UNet	Enc-Dec + skip	No	No	Linear
UNet Residual	Enc-Dec + skip	Yes	No	Linear
Attention UNet	Enc-Dec + skip	No	AG	Linear
DnCNN	Sequential	Yes	No	Clamp
Denoising AE	Enc-Dec (no skip)	No	No	Tanh
RIDNet	Feature + EAB	Yes	Channel	Clamp
Pix2Pix Gen.	UNet-like (7L)	Config.	No	Tanh

TABLE I: Summary of implemented architectures. AG = Attention Gates; Channel = Squeeze-and-Excitation channel attention.

J. Architecture Summary

V. TRAINING

A. Training Pipeline

The training loop is implemented in `trainer.py` and supports all architectures through a unified interface. The function `run_training` orchestrates the entire process: learning rate warmup, train/validation alternation, checkpointing, early stopping, and logging.

1) *Mixed Precision (AMP)*: All experiments (except the Denoising Autoencoder) use PyTorch Automatic Mixed Precision [10]: forward pass operations execute in FP16 via `torch.autocast`, while the loss is always computed in FP32 for numerical stability. A `GradScaler` manages gradient rescaling. Validation is **always** performed in full precision, regardless of AMP settings, to avoid instabilities caused by BatchNorm in eval mode interacting with FP16.

2) *Learning Rate Warmup*: The first `warmup_epochs` (typically 5–8) use linear warmup of the learning rate from a small initial value to the target rate. The main scheduler (CosineAnnealing or StepLR) activates only after warmup completion.

3) *Gradient Clipping*: Gradients are clipped via `clip_grad_norm_` with a configurable maximum norm (typically 0.5–1.0). Under AMP, gradients are first unscaled by the `GradScaler` before clipping.

4) *Early Stopping*: The patience counter increments only at validation epochs where the validation loss does not improve. This allows configuring validation at intervals (*e.g.*, every 2 epochs) without penalizing the patience count.

5) *OOM Error Handling*: On Out-of-Memory errors during training, the system: (1) saves an emergency checkpoint, (2) frees CUDA memory, and (3) prints diagnostic suggestions (reduce batch/patch size, enable AMP).

B. Pix2Pix Training

Pix2Pix follows the GAN paradigm with alternating discriminator and generator updates.

1) *Discriminator Training*: The discriminator receives image pairs (conditional + target):

- **Real pairs**: $(x_{\text{degraded}}, x_{\text{clean}}) \rightarrow \text{label } 1$
- **Fake pairs**: $(x_{\text{degraded}}, G(x_{\text{degraded}})) \rightarrow \text{label } 0$

$$\mathcal{L}_D = \frac{1}{2} (\text{BCE}(D(x, y_{\text{real}}), 1) + \text{BCE}(D(x, y_{\text{fake}}), 0)) \quad (16)$$

2) *Generator Training*: The generator loss combines adversarial and reconstruction components:

$$\mathcal{L}_G = \text{BCE}(D(x, G(x)), 1) + \lambda_{L1} \cdot \mathcal{L}_{\text{recon}}(G(x), y) \quad (17)$$

where $\lambda_{L1} = 150$ and the reconstruction loss is the Combined Loss (L1 + SSIM). The high λ_{L1} balances the adversarial component, which tends to dominate in early training phases.

3) *Noise Diagnostics*: In residual learning mode, statistics on predicted noise are monitored: mean, standard deviation, maximum value, outlier percentage (values exceeding the clamp threshold), and percentage of clipped output pixels.

C. Experiment Configurations

Table II summarizes the main hyperparameters for each model’s best experiment. All experiments use DIV2K with Gaussian noise $\sigma = 100$ unless otherwise noted.

D. Implementation Notes

- **Optimizers**: UNet variants use AdamW (weight_decay = 10^{-5}). DnCNN uses Adam with **no weight decay** (found empirically beneficial). RIDNet uses Adam (weight_decay = 10^{-4}). Pix2Pix uses Adam with $\beta_1 = 0.5$.
- **Schedulers**: CosineAnnealingLR for most models; StepLR (step=1, $\gamma = 0.96$) for DnCNN; no scheduler for Pix2Pix generator.
- **Attention UNet bottleneck variants**: Use GroupNorm instead of BatchNorm, shallower perceptual layers (relu1_2 + relu2_2), and reduced perceptual weight ($\gamma = 0.05$).
- **RIDNet specifics**: 8 EAB blocks with residual_scale = 0.1, 96 feature channels (vs. 64 default), and stricter gradient clipping (norm = 0.5).
- **Dithering experiments**: Two additional experiments were conducted with 2-bit random dithering using the base UNet (both bilinear and transposed convolution variants).

VI. EVALUATION METHODOLOGY

A. Metrics

Model performance is evaluated using four quantitative metrics, computed on images normalized to $[0, 1]$.

1) *PSNR (Peak Signal-to-Noise Ratio)*:

$$\text{PSNR} = 20 \cdot \log_{10} \left(\frac{\text{MAX}}{\sqrt{\text{MSE}}} \right) \quad [\text{dB}] \quad (18)$$

where MAX = 1.0. Higher values indicate greater fidelity to ground truth.

2) *SSIM (Structural Similarity Index)*: SSIM [8] is computed with an 11×11 window via `F.avg_pool2d`:

$$\text{SSIM}(x, y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)} \quad (19)$$

with $C_1 = (0.01)^2$ and $C_2 = (0.03)^2$. Values closer to 1 indicate higher structural similarity.

Model	Loss ($\alpha/\beta/\gamma$)	LR	Optimizer	Epochs	Batch	Patch	Warmup	Patience	Scheduler
UNet (bilinear)	L1+SSIM (0.84/0.16)	1e-4	AdamW	36	16	128	5	5	Cosine
UNet Res. (bilinear)	L1+SSIM+VGG (0.6/0.25/0.15)	1e-4	AdamW	36	16	128	5	5	Cosine
Att. UNet (btlk)	L1+SSIM+VGG (0.65/0.3/0.05)	4e-5	AdamW	36	16	128	8	5	Cosine
DnCNN (17L, 64F)	L1+SSIM (0.85/0.15)	1e-3	Adam	100	32	96	—	10	Step
Denoising AE	L1+SSIM (0.84/0.16)	1e-4	Adam	200	40	128	2	15	Cosine
RIDNet (8 EAB)	L1+SSIM (0.8/0.2)	1e-4	Adam	150	8	256	5	10	Cosine
Pix2Pix (TODO)	GAN+L1+SSIM	G:2e-4	Adam	150	8	384	—	25	—

TABLE II: Training hyperparameters for each model’s best configuration. Gaussian noise $\sigma = 100$ on DIV2K. Loss weights shown as α/β (L1/SSIM) or $\alpha/\beta/\gamma$ (L1/SSIM/Perceptual).

3) *MAE and MSE:* Mean Absolute Error and Mean Squared Error, computed pixel-wise via `F.l1_loss` and `F.mse_loss` respectively.

B. Full-Resolution Inference

DIV2K images have resolution $\sim 2000 \times 2000$ pixels, too large for a single forward pass. Inference is performed via **sliding window** with weighted blending.

1) *Sliding Window:* The image is partitioned into overlapping patches of size `patch_size` (typically 256) with overlap pixels (typically 32–64) of overlap:

$$\text{stride} = \text{patch_size} - \text{overlap} \quad (20)$$

2) *Hann Window Blending:* To avoid boundary artifacts between patches, each output is weighted with a 2D Hann window, obtained as the outer product of two 1D windows:

$$w(i, j) = w_H(i) \cdot w_H(j), \quad w_H(k) = 0.5 - 0.5 \cos\left(\frac{2\pi k}{N-1}\right) \quad (21)$$

The final output for each pixel is the weighted average of contributions from all overlapping patches:

$$I_{\text{out}}(i, j) = \frac{\sum_p w_p \cdot O_p(i, j)}{\sum_p w_p} \quad (22)$$

This produces smooth, natural transitions between adjacent patches.

C. Evaluation Protocol

The `ImageRestorationEvaluator` class orchestrates evaluation on the full dataset:

- 1) For each (degraded, clean) pair: sliding-window inference followed by per-image metric computation.
- 2) Aggregation: mean and standard deviation across all images.
- 3) Reporting: JSON export with per-image and aggregate metrics, identification of best and worst cases.

All models are evaluated on the same 10 validation images (indices 0801–0810 from DIV2K), enabling direct per-image comparisons across architectures.

Model	PSNR (dB)	SSIM	MAE	Epochs
RIDNet (8 EAB)	25.83 ± 2.41	0.725 ± 0.081	0.0369	44
DnCNN (17L, 64F)	25.60 ± 2.24	0.694 ± 0.066	0.0385	86
Att. UNet (btlk)	24.76 ± 1.98	0.678 ± 0.056	0.0424	34
UNet (bilinear)	24.53 ± 1.91	0.639 ± 0.052	0.0444	18
DAE (upsample)	24.27	0.660	0.0423	200
UNet Res. (bilinear)	23.66 ± 1.81	0.594 ± 0.040	0.0483	10
Pix2Pix	[TODO — Daniele]			
NAFNet	[TODO — Daniele]			
Swin2SR	[TODO — Daniele]			

TABLE III: Main results on DIV2K validation (Gaussian $\sigma = 100$). Best values in bold. Models ranked by PSNR. DAE trained on $\sigma = 100$ with upsample+conv decoder.

VII. EXPERIMENTAL RESULTS

All experiments were conducted on DIV2K [1] with Gaussian noise $\sigma = 100$. Models were trained on 800 training images and evaluated on 10 validation images (indices 0801–0810) at full resolution using sliding-window inference with Hann blending.

A. Quantitative Comparison

Table III presents the main results for the best configuration of each model.

Key observations:

- RIDNet achieves the best overall performance across all metrics, with a margin of +0.23 dB PSNR and +0.031 SSIM over DnCNN.
- Both top models (RIDNet, DnCNN) employ residual learning and operate without spatial downsampling, preserving full resolution throughout the network.
- The Attention UNet with bottleneck attention and Group-Norm outperforms the base UNet by +0.23 dB, demonstrating the benefit of selective feature weighting.
- The Denoising Autoencoder, despite lacking skip connections, achieves competitive SSIM (0.660), suggesting the bottleneck forces learning of global structure.
- UNet Residual underperforms the direct-prediction UNet, likely due to early stopping (only 10 epochs) preventing convergence.

B. Training Efficiency

The UNet variants offer the best efficiency (PSNR per training hour) due to their moderate computational cost. RIDNet

Model	Time (h)	Memory (MB)	PSNR/h
RIDNet (8 EAB)	3.41	30.83	7.57
DnCNN (17L)	3.01	17.16	8.51
Att. UNet (btlnk)	1.63	497.7	15.19
UNet (bilinear)	0.84	273.2	29.20
DAE (upsample)	~11.1	—	2.19
UNet Res. (bilinear)	0.47	275.7	50.34

TABLE IV: Training efficiency comparison. PSNR/h indicates PSNR achieved per hour of training. Memory is peak GPU allocation during training.

Layers	Features	LR	WD	PSNR	SSIM
20	128	1e-3	0	24.86	0.635
17	64	1e-3	0	25.60	0.694
20	64	1e-4	0	22.86	0.518

TABLE V: DnCNN ablation study. Best configuration uses fewer layers, smaller feature count, and higher learning rate. The 10 $\times$ LR reduction in the third run severely degrades performance.

and DnCNN require longer training but achieve superior final quality. The DAE requires 200 epochs (~11 hours) due to the absence of skip connections, which slows information flow.

C. Ablation Studies

1) *DnCNN: Depth, Features, and Learning Rate:* The DnCNN ablation (Table V) reveals three key findings: (1) A smaller network (17 layers, 64 features) outperforms the larger one (20 layers, 128 features), suggesting that over-parameterization hurts for this noise level. (2) Zero weight decay is critical—any regularization harms the residual learning capacity. (3) Learning rate is the single most impactful factor: reducing LR from 10^{-3} to 10^{-4} drops PSNR by 2.74 dB.

2) *RIDNet: Number of EAB Blocks:* The RIDNet ablation (Table VI) shows that: (1) Too few EABs (4) limit representational capacity, especially with small patches (64). (2) The 12-EAB model achieves good results with only 16 epochs, suggesting larger models converge faster but are cut short by early stopping. (3) The optimal 8-EAB configuration with 96 features and 256-pixel patches provides the best balance of capacity and trainability. Gradient clipping (norm = 0.5) proved essential for training stability with larger EAB counts.

3) *Denoising Autoencoder: Decoder Architecture:* The DAE ablation (Table VII) demonstrates that replacing ConvTranspose2d with bilinear upsampling followed by a learned convolution significantly improves structural quality (SSIM +0.086) by eliminating checkerboard artifacts in the decoder output.

4) *Attention UNet: Attention Configuration and Upsampling:* The Attention UNet ablation (Table VIII) reveals a surprising finding: applying attention gates at *all* decoder levels actually *hurts* performance compared to bottleneck-only attention. Combined with GroupNorm (which provides better gradient flow in small-batch settings), the bottleneck variant improves PSNR by +2.36 dB over the all-attention variant. This suggests that excessive attention modulation in earlier decoder stages can disrupt low-level feature propagation.

EABs	Features	Patch	Epochs	PSNR	SSIM
4	64	64	47	23.27	0.552
12	64	256	16	24.97	0.677
8	96	256	44	25.83	0.725

TABLE VI: RIDNet ablation on number of EAB blocks. The optimal configuration balances model capacity (8 EAB, 96 features) with sufficient training duration.

Decoder	Batch	Epochs	PSNR	SSIM
ConvTranspose (F=64, b=10)	10	150	24.03	0.598
ConvTranspose (F=64, b=40)	40	95	23.74	0.574
Upsample+Conv (F=64, b=40)	40	200	24.27	0.660

TABLE VII: Denoising Autoencoder ablation ($\sigma = 100$). Bilinear upsampling + convolution eliminates checkerboard artifacts and improves SSIM by +0.086 over the same batch-size ConvTranspose variant.

5) *UNet on Dithering Degradation:* On the dithering degradation task (Table IX), the UNet with transposed convolution upsampling outperforms the bilinear variant by +1.21 dB, despite having fewer feature channels (64 vs. 96). This contrasts with Gaussian denoising where both modes perform similarly, suggesting that learned upsampling kernels are particularly beneficial for structured degradation patterns.

D. Visual Comparison

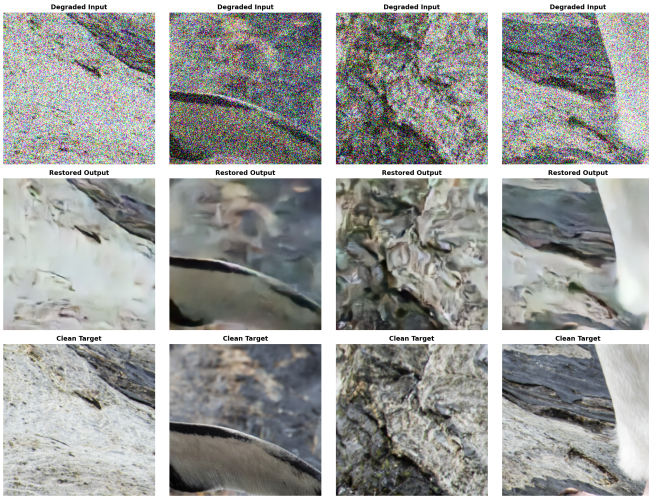
Fig. 1 shows qualitative results from the best run of each model. Key visual observations:

- **RIDNet** produces the sharpest restorations with best preservation of edges and textures, thanks to its multi-scale dilated convolutions and channel attention.
- **DnCNN** achieves similarly clean results but tends to slightly over-smooth fine textures compared to RIDNet.
- **Attention UNet** restores large-scale structure well but produces slightly blurrier outputs than the sequential models.
- **Denoising Autoencoder** produces the smoothest results, confirming that the information bottleneck (no skip connections) limits high-frequency detail preservation.

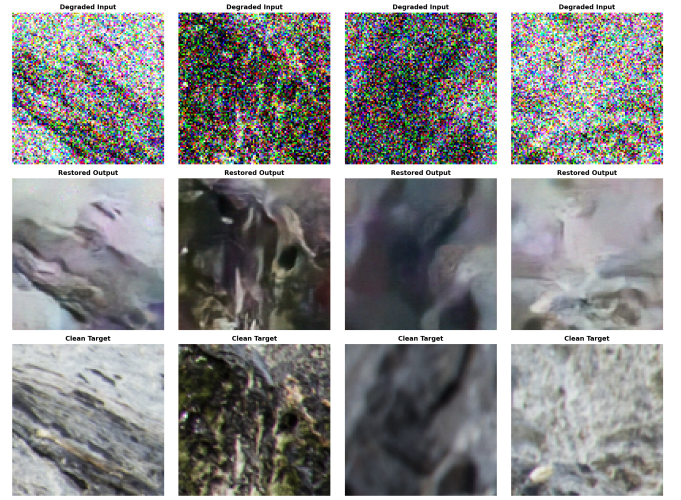
E. Training Dynamics

Fig. 2 shows training dynamics for representative models. Several patterns emerge:

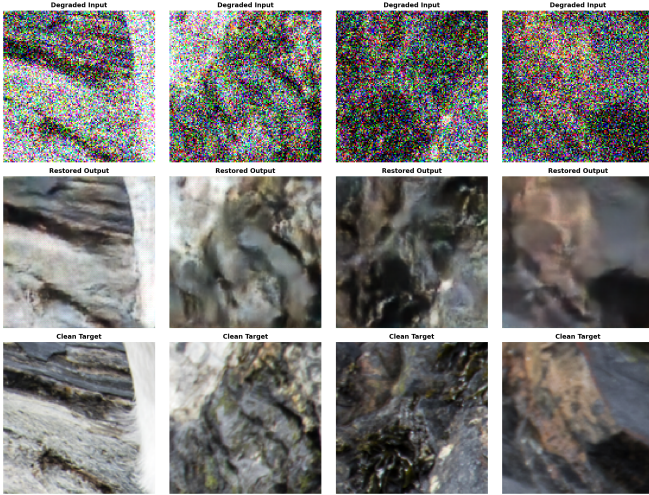
- **Skip connections accelerate convergence:** UNet variants converge in 10–36 epochs, while skip-free architectures (DAE) require 150–200 epochs.
- **RIDNet combines fast convergence with high quality:** Despite its depth (8 EAB blocks), gradient clipping and cosine scheduling enable smooth convergence in 44 epochs.
- **Warmup is critical for complex architectures:** The Attention UNet and RIDNet both benefit from 5–8 warmup epochs; without warmup, initial instability can trigger early stopping prematurely.



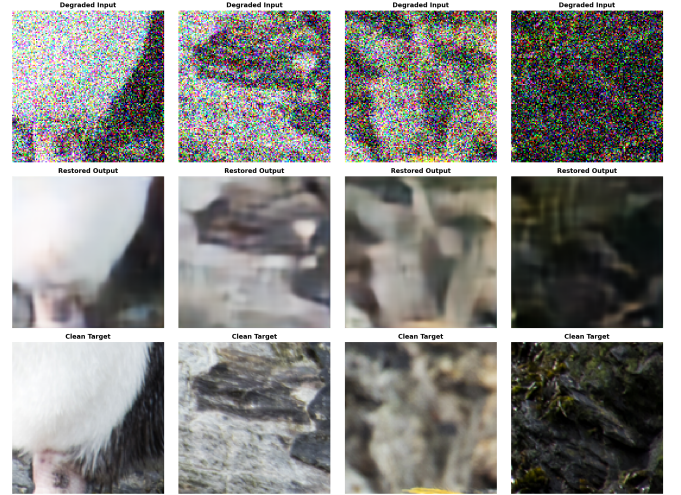
(a) RIDNet (8 EAB) — PSNR 25.83 dB, SSIM 0.725



(b) DnCNN (17L) — PSNR 25.60 dB, SSIM 0.694



(c) Attention UNet (bottleneck) — PSNR 24.76 dB, SSIM 0.678



(d) Denoising Autoencoder (upsample) — PSNR 24.27 dB, SSIM 0.660

Fig. 1: Visual comparison of inference results across models. Each sub-figure shows degraded input (left), model output (center), and ground truth (right) for a validation image from DIV2K. RIDNet and DnCNN preserve the most fine-grained details, while the DAE produces smoother but less detailed outputs due to the information bottleneck.

Variant	Upsample	Attention	Norm	PSNR	SSIM
bilinear_v2	Bilinear	All	BN	22.40	0.573
upsample	TransConv	All	BN	21.81	0.607
upsample_btlnk	TransConv	Bottleneck	GN	24.76	0.678
bilinear_btlnk	Bilinear	Bottleneck	GN	24.64	0.671

TABLE VIII: Attention UNet ablation. Bottleneck-only attention with GroupNorm dramatically outperforms all-level attention with BatchNorm. BN = BatchNorm, GN = GroupNorm.

Variant	Features	PSNR	SSIM
Bilinear (F=96)	96	27.43	0.763
TransConv (F=64)	64	28.64	0.792

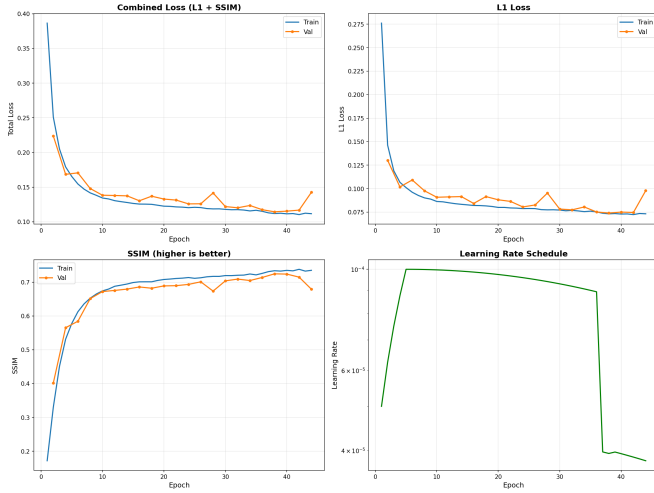
TABLE IX: UNet performance on 2-bit random dithering. Transposed convolution upsampling outperforms bilinear despite fewer features, likely due to learned upsampling kernels better suited to the structured dithering patterns.

- **Learning rate is the dominant factor for DnCNN:** The successful run ($LR=10^{-3}$) converges steadily, while the failed run ($LR=10^{-4}$) plateaus early and runs for 10+ hours without meaningful improvement.

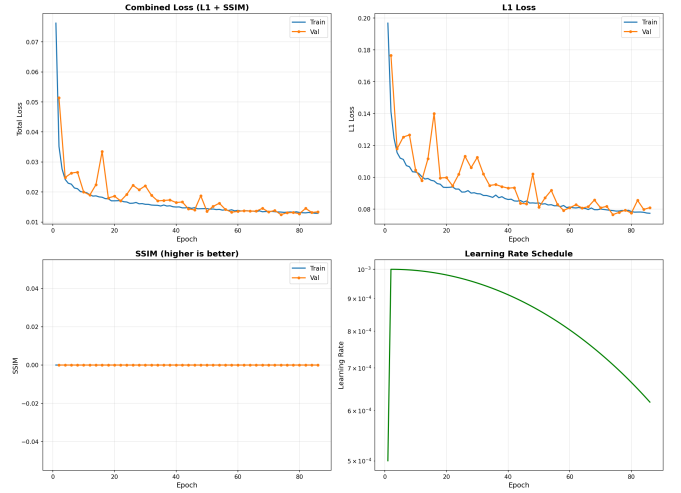
F. Per-Image Analysis

Table X reveals consistent per-image patterns:

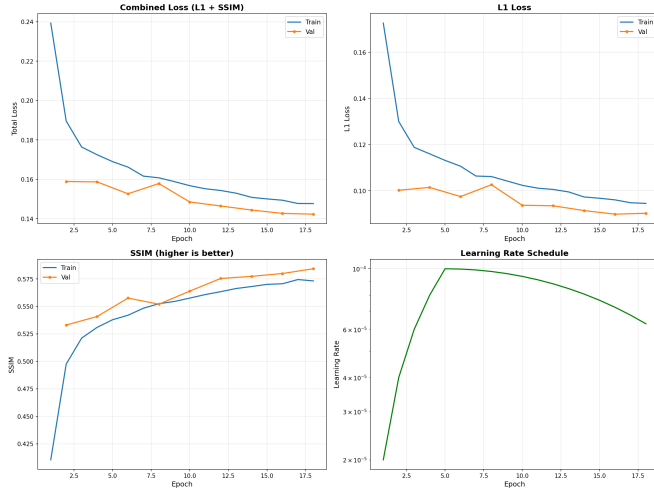
- **Image 0803** is the easiest for all models (PSNR up to 30.14 dB with RIDNet), likely due to large smooth regions where noise removal is straightforward.
- **Image 0807** is the hardest (PSNR ~ 21 – 22 dB across all models), likely containing high-frequency textures that are difficult to distinguish from noise at $\sigma = 100$.
- The performance gap between models is relatively con-



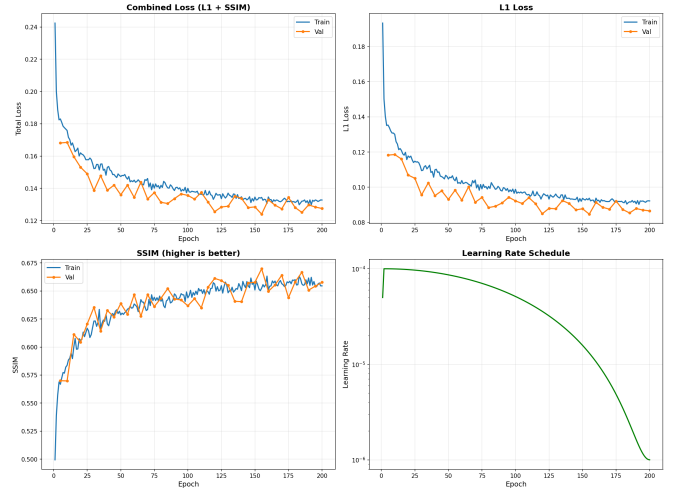
(a) RIDNet (8 EAB) — smooth convergence in 44 epochs



(b) DnCNN (17L) — slower convergence, 86 epochs



(c) UNet (bilinear) — rapid initial convergence, early stopping at 18 epochs



(d) Denoising AE — slow convergence requiring 200 epochs

Fig. 2: Training curves for representative models. The UNet family converges quickly (10–36 epochs) due to skip connections. DnCNN requires more epochs but benefits from the higher learning rate. The DAE converges slowest due to the information bottleneck forcing gradual feature learning.

sistent across images (RIDNet > DnCNN > Att.UNet > UNet), suggesting architectural advantages are content-independent.

- DnCNN occasionally outperforms RIDNet on specific images (0809, 0810), indicating that the models capture complementary features.

G. Pending Results

[TODO — Daniele]: The following models require experimental results to be added:

- **Pix2Pix:** Conditional GAN results on Gaussian noise $\sigma = 100$. Expected metrics: PSNR, SSIM, MAE, plus GAN-specific metrics (FID, discriminator accuracy).
- **NAFNet:** Nonlinear Activation Free Network results and comparison with the other architectures.

- **Swin2SR:** Transformer-based restoration results, particularly interesting for comparison with purely convolutional approaches.

VIII. CONCLUSION

This work presented a comprehensive comparative study of seven CNN-based architectures for image denoising under severe Gaussian noise ($\sigma = 100$). All models were implemented within a unified, modular framework and evaluated under identical conditions on the DIV2K dataset.

A. Key Findings

- 1) **RIDNet achieves state-of-the-art performance** within our comparison (PSNR = 25.83 dB, SSIM = 0.725), demonstrating that the combination of dilated convolutions for multi-scale feature capture and channel atten-

Image	RIDNet	DnCNN	Att.UNet	UNet	DAE
0801	25.67	25.29	24.36	24.30	—
0802	24.97	24.57	23.79	23.53	—
0803	30.14	29.29	27.86	26.91	—
0804	24.33	24.09	23.33	22.95	—
0805	25.55	24.89	24.24	24.27	—
0806	24.43	24.11	23.40	23.22	—
0807	22.28	22.24	21.33	21.42	—
0808	27.21	26.92	26.01	25.36	—
0809	27.05	27.16	26.46	25.49	—
0810	26.63	27.48	26.81	25.88	—
Mean	25.83	25.60	24.76	24.53	24.27

TABLE X: Per-image PSNR (dB) comparison across models. Bold = best per image; italic = worst image across all models. Image 0803 is consistently easiest (smooth, low-frequency content); 0807 is hardest (high-frequency textures).

tion for adaptive feature weighting is highly effective for heavy noise removal.

- 2) **Residual learning is consistently beneficial** for sequential architectures (DnCNN, RIDNet), while encoder-decoder architectures show mixed results—the UNet with residual learning underperformed its direct-prediction counterpart due to insufficient training.
- 3) **Attention mechanisms require careful design:** Bottleneck-only attention with GroupNorm outperforms all-level attention with BatchNorm by +2.36 dB, suggesting that over-application of attention gates can disrupt feature propagation in skip connections.
- 4) **Upsampling strategy matters for structured degradations:** Learned transposed convolutions outperform bilinear upsampling on dithering (+1.21 dB) but show minimal difference on Gaussian noise, indicating that upsampling choice should be degradation-aware.
- 5) **Training hyperparameters can dominate architecture choice:** A $10\times$ learning rate reduction degrades DnCNN by 2.74 dB—more than the gap between the best and worst architectures. Gradient clipping (norm = 0.5) proved essential for RIDNet stability.
- 6) **Skip connections dramatically accelerate convergence:** UNet variants converge in 10–36 epochs vs. 200 epochs for the skip-free Denoising Autoencoder, highlighting the importance of direct gradient paths for efficient training.

B. Architectural Insights

The results suggest a hierarchy of architectural components by importance:

- 1) *Learning paradigm* (residual vs. direct) — largest impact for sequential networks;
- 2) *Multi-scale feature capture* (dilated convolutions, large receptive fields) — enables capturing noise at different frequencies;
- 3) *Channel attention* (SE blocks) — provides adaptive feature selection;

- 4) *Skip connections* — critical for training efficiency and high-frequency detail preservation;
- 5) *Upsampling strategy* — secondary for Gaussian noise but important for structured degradations.

C. Pending Work

[TODO — Daniele]: The following contributions are pending integration:

- **Pix2Pix** experimental results and analysis of GAN-based denoising advantages/limitations (perceptual quality vs. metric optimization);
- **NAFNet** [12] results demonstrating the viability of activation-free architectures;
- **Swin2SR** [13] results comparing transformer-based approaches with purely convolutional methods.

These additions will enable a complete comparison spanning CNNs, GANs, and Transformers for the image restoration task.

D. Future Directions

Potential extensions include: (1) evaluating on real-world noise distributions beyond synthetic Gaussian noise; (2) exploring ensemble methods combining the complementary strengths of RIDNet and DnCNN observed in per-image analysis; (3) investigating self-supervised denoising approaches that do not require clean reference images; and (4) extending the framework to other restoration tasks (super-resolution, deblurring, JPEG artifact removal).

REFERENCES

- [1] E. Agustsson and R. Timofte, “NTIRE 2017 Challenge on Single Image Super-Resolution: Dataset and Study,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, 2017, pp. 1122–1131.
- [2] O. Ronneberger, P. Fischer, and T. Brox, “U-Net: Convolutional Networks for Biomedical Image Segmentation,” in *Proc. Med. Image Comput. Comput.-Assisted Intervention (MICCAI)*, 2015, pp. 234–241.
- [3] K. Zhang, W. Zuo, Y. Chen, D. Meng, and L. Zhang, “Beyond a Gaussian Denoiser: Residual Learning of Deep CNN for Image Denoising,” *IEEE Trans. Image Process.*, vol. 26, no. 7, pp. 3142–3155, 2017.
- [4] S. Anwar and N. Barnes, “Real Image Denoising with Feature Attention,” in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2019, pp. 3155–3164.
- [5] P. Isola, J.-Y. Zhu, T. Zhou, and A. A. Efros, “Image-to-Image Translation with Conditional Adversarial Networks,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2017, pp. 1125–1134.
- [6] O. Oktay *et al.*, “Attention U-Net: Learning Where to Look for the Pancreas,” in *Proc. Med. Image Learn. Less Labels Imperfect Data (MIDL)*, 2018.
- [7] J. Johnson, A. Alahi, and L. Fei-Fei, “Perceptual Losses for Real-Time Style Transfer and Super-Resolution,” in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2016, pp. 694–711.
- [8] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, “Image Quality Assessment: From Error Visibility to Structural Similarity,” *IEEE Trans. Image Process.*, vol. 13, no. 4, pp. 600–612, 2004.
- [9] J. Hu, L. Shen, and G. Sun, “Squeeze-and-Excitation Networks,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2018, pp. 7132–7141.
- [10] P. Micikevicius *et al.*, “Mixed Precision Training,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2018.
- [11] R. W. Floyd and L. Steinberg, “An Adaptive Algorithm for Spatial Greyscale,” *Proc. Soc. Inf. Disp.*, vol. 17, no. 2, pp. 75–77, 1976.
- [12] L. Chen *et al.*, “Simple Baselines for Image Restoration,” in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2022, pp. 17–33.

- [13] M. V. Conde, U.-J. Choi, M. Burber, and R. Timofte, “Swin2SR: SwinV2 Transformer for Compressed Image Super-Resolution and Restoration,” in *Proc. Eur. Conf. Comput. Vis. Workshops (ECCVW)*, 2022.